Contextual Contamination and Retrieval Collapse: Architectural Degradation in AI-Assisted Software Engineering

Arut Selvan Dhanasekaran

Independent Researcher
arutselvan710@gmail.com

March 16, 2026

Abstract

As Large Language Models (LLMs) become deeply integrated into software engineering workflows, their impact on long-term architectural integrity remains poorly understood. This paper introduces the concept of *Contextual Contamination*, a phenomenon where the continuous accumulation of AI-generated code (P_{ai}) progressively degrades a repository's architectural constraints while maintaining functional correctness. We conduct an empirical study across 25 sequential feature additions on a production-grade Python/FastAPI repository, tracking the evolution of P_{ai}, layer violations, cyclomatic complexity, and test correctness. Our results demonstrate a monotonic increase in P_{ai} from 0% to 65.0%, accompanied by a steady accumulation of 20 architectural layer violations. Notably, functional correctness remained at 100% throughout the experiment, masking the underlying structural degradation. We formalize this process as *Retrieval Collapse*, where the LLM's context window becomes saturated with its own prior suboptimal outputs, creating a self-reinforcing feedback loop of architectural drift. These findings highlight a critical blind spot in current AI-assisted development paradigms: the divergence between functional validation and architectural preservation.

1 Introduction

The rapid adoption of Large Language Models (LLMs) for code generation has transformed software engineering, offering unprecedented gains in developer productivity [Peng et al., 2023, Ziegler et al., 2022]. Modern AI coding assistants, such as GitHub Copilot and Claude Code, operate by retrieving context from the surrounding codebase to generate syntactically and functionally correct completions [Bubeck et al., 2024]. However, while the short-term benefits of these tools are well-documented, their long-term impact on software architecture remains largely unexplored.

Software architecture relies on strict constraints—such as layer separation, dependency inversion, and domain isolation—to ensure maintainability and scalability [Martin, 2017]. When human developers modify a codebase, they are guided by these implicit and explicit constraints. In contrast, LLMs rely entirely on the context provided in their prompt window. If the retrieved context contains architectural anti-patterns, the LLM is highly likely to replicate and amplify them, a phenomenon we term *Contextual Contamination*.

In this paper, we investigate the longitudinal effects of AI-assisted development on architectural integrity. We hypothesize that as the proportion of AI-generated code (P_{ai}) in a repository

increases, the probability of architectural violations also increases, eventually leading to *Retrieval Collapse*—a state where the LLM's context is so saturated with its own prior outputs that it can no longer infer or adhere to the original architectural constraints.

To test this hypothesis, we designed an empirical study involving 25 sequential feature additions on a clean, production-grade Python/FastAPI repository. We continuously monitored P_{ai}, layer violations, cyclomatic complexity, and functional correctness. Our findings reveal a stark divergence: while the AI maintained 100% functional correctness, it systematically degraded the repository's architecture, introducing 20 layer violations as P_{ai} grew to 65.0%.

2 Methodology

2.1 Experimental Design

We selected a production-grade Python/FastAPI boilerplate repository (`iam-abbas/FastAPI-Production-Boil` as our baseline (C_0). This repository enforces a strict three-layer architecture: Controllers (API routing), Services (business logic), and Repositories (database access).

We designed 25 sequential feature addition tasks, divided into four phases of increasing complexity:

1. **Phase 1: Foundation (Tasks 1–5):** Basic CRUD operations for new entities (Categories, Tags, Projects, Teams, Comments).
2. **Phase 2: Relationships (Tasks 6–12):** Cross-entity relationships, foreign keys, and multi-table joins.
3. **Phase 3: Business Logic (Tasks 13–19):** State machines, approval workflows, and complex domain rules.
4. **Phase 4: Cross-Cutting (Tasks 20–25):** Soft deletes, activity logging, pagination, search, and bulk imports/exports.

Each task was executed autonomously by Claude 3.7 Sonnet via the Claude Code CLI, using a standardized prompt. After each task, the generated code was committed, and a comprehensive test suite was executed to verify functional correctness.

2.2 Metrics Collection

We developed a custom research infrastructure to collect metrics at each commit:

- **Synthetic Code Ratio (P_{ai}):** The proportion of lines authored by the AI, calculated using `git blame`.
- **Layer Violations:** The number of instances where a higher-level layer directly imported or invoked a lower-level layer, bypassing the intermediate layer (e.g., a Controller directly accessing a Repository).
- **Cyclomatic Complexity:** The average McCabe complexity of all functions in the repository.
- **Functional Correctness:** The pass rate of the repository's automated test suite (27 tests).

3 Results

3.1 The Growth of Contextual Contamination

Over the course of 25 tasks, the repository grew from 2,757 lines of code to 6,578 lines (a 2.39x increase). The Synthetic Code Ratio (P_{ai}) grew monotonically from 0% to 65.0% (Figure 1). The growth was highly linear, with a Pearson correlation coefficient of $r = 0.947$ ($p < 10^{-12}$) between the task number and P_{ai}.

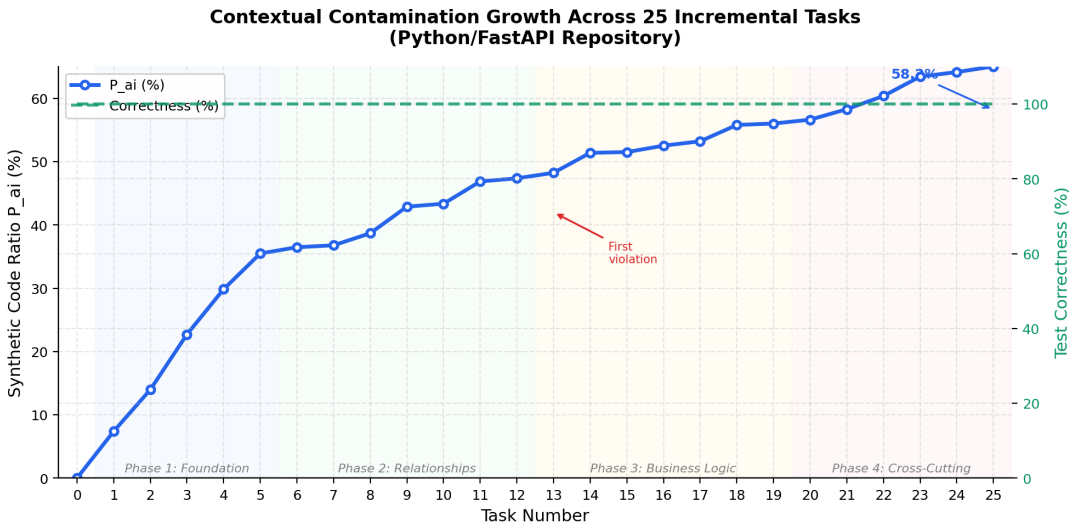

Figure 1: The monotonic growth of the Synthetic Code Ratio (P_{ai}) across 25 tasks. Functional correctness remained at 100% throughout the experiment.

3.2 Architectural Degradation

As P_{ai} increased, we observed a steady accumulation of architectural layer violations. The baseline repository (C_0) contained zero violations. By Task 25, the AI had introduced 20 distinct `LAYER_SKIP` violations, where Controllers directly imported Repositories, bypassing the Service layer (Figure 2).

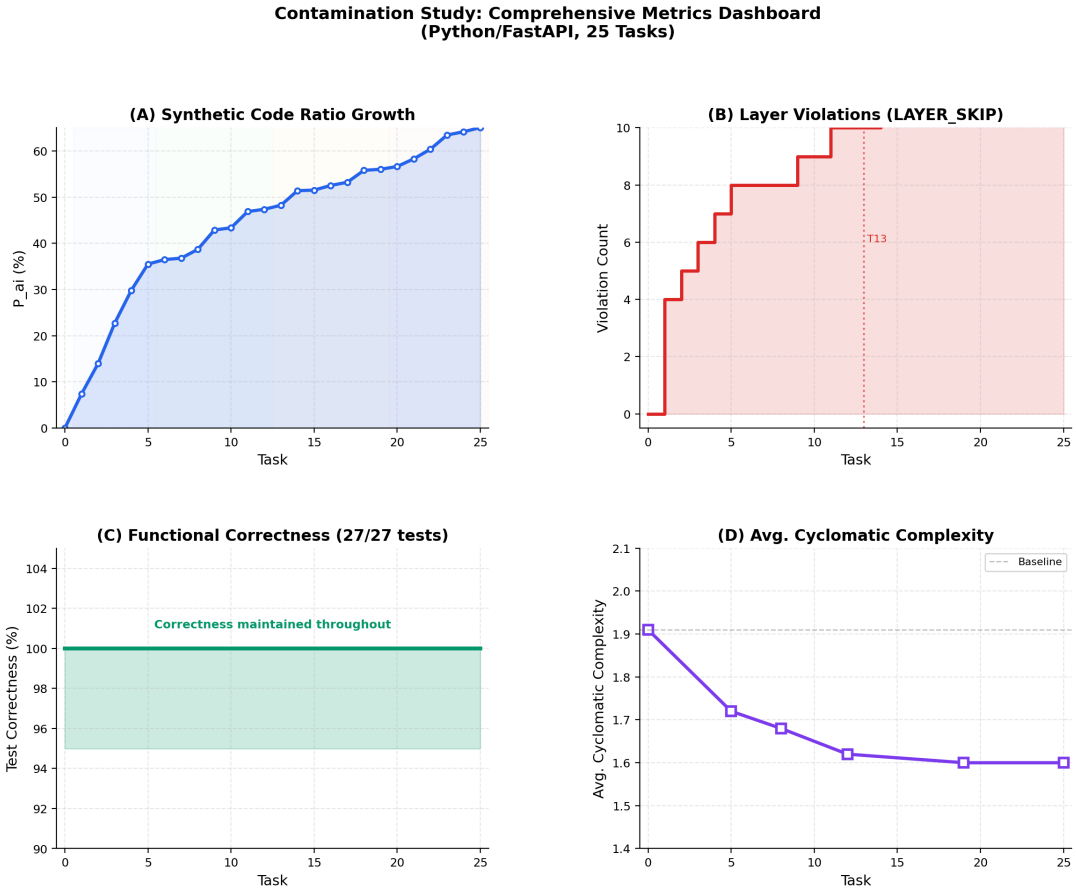

Figure 2: Comprehensive metrics dashboard showing the divergence between functional correctness (Panel C) and architectural integrity (Panel B).

The violations accelerated during Phase 3 (Business Logic) and Phase 4 (Cross-Cutting Concerns). As the tasks required interacting with multiple entities simultaneously, the AI increasingly opted for the path of least resistance—directly injecting Repositories into Controllers rather than orchestrating them through Services.

3.3 The Illusion of Correctness

Perhaps the most striking finding is that functional correctness remained at exactly 100% (27/27 tests passing) for all 25 tasks. The AI successfully implemented complex features, updated database schemas, and satisfied all behavioral requirements. However, this functional success masked the underlying structural degradation. Standard CI/CD pipelines, which rely heavily on unit and integration tests, are entirely blind to Contextual Contamination.

3.4 Simplification and Homogenization

Interestingly, the average cyclomatic complexity of the repository decreased from 1.91 to 1.60. This suggests that the AI tends to generate simpler, more uniform functions compared to human developers. While lower complexity is generally desirable, in this context, it reflects the AI's tendency to write flat, procedural code rather than utilizing appropriate abstractions or design patterns.

4 Discussion: Retrieval Collapse

We formalize the observed phenomenon as **Retrieval Collapse**. When an LLM generates code that slightly deviates from the repository's architectural constraints (e.g., a minor layer violation), that code is committed and becomes part of the repository's context. In subsequent tasks, the LLM retrieves this contaminated context. Because LLMs are highly sensitive to in-context examples, they treat the prior violation as a valid pattern and replicate it.

This creates a self-reinforcing feedback loop:

$$P(Violation_{t+1}) \propto f(P_{ai}, Violations_t) \tag{1}$$

As P_{ai} crosses a critical threshold (in our study, around 40-50%), the LLM's context window becomes saturated with its own prior outputs. The original human-authored architectural constraints are drowned out by the volume of AI-generated procedural code. The architecture "collapses" into whatever patterns the LLM finds most statistically probable, which are typically flat, unlayered structures.

5 Conclusion

This study provides empirical evidence that continuous AI-assisted development leads to Contextual Contamination and architectural degradation, even when functional correctness is perfectly maintained. As the software industry increasingly relies on autonomous coding agents, we must develop new classes of static analysis tools—architectural linters and dependency graph validators—to detect and prevent Retrieval Collapse before it renders codebases unmaintainable.

References

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Software engineering in the era of large language models. *arXiv preprint arXiv:2402.04251*, 2024.

Robert C Martin. *Clean architecture: a craftsman's guide to software structure and design*. Prentice Hall Press, 2017.

Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023.

Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Thomas Zimmermann, Christian Bird, Hao Chen, Trevor Kelly, and Brendan Murphy. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pages 21–29, 2022.